



An-Najah National University

Shield Gates

Secure Email System with Mail Gateway Filtering

Technical Documentation

The Supervisor:

Dr. Othman M. Othman & A. Dr. Prof Jehad M. Hamamreh

By:

Rama Abu-Haweleha	12219068
Rana Shiqer	12218105

Appendix C

This appendix delineates the step-by-step procedures required to clone the deployment repository, construct the isolated containerized environment using Docker, and execute the core architectural components of the ****Shields Gate**** Mail Gateway. Furthermore, it addresses critical configurations for enterprise routing, environment customization, and infrastructure adaptability.

1. Repository Replication and Context Alignment

To initialize the deployment, the target platform must mirror the version-controlled repository. Execute the following commands to navigate to the host deployment context, replicate the codebase, and align the context directories for the Docker daemon:

```
git clone https://github.com/RamaAbuHaweleha01/MGW.git MGW-Docker
cd MGW-Docker
```

Consolidate structural components into the localized build context

```
cp -r ~/MGW/ ./MGW
cp -r ~/CAPEv2/ ./CAPEv2
```

Establish the persistent directory for Postfix configuration lifecycle

```
mkdir -p postfix_config
cp ~/MGW/MGW-Postfix-Config-File/* ./postfix_config/
```

2 Core Configuration and Network Range Tailoring

Before building the container image, the network boundaries and external mail routing must be validated to ensure alignment with the targeted Security Operations Center (SOC) or lab layout.

2.1 Mail Server Routing and Access Control Lists (`main.cf`)

Open the localized configuration (`postfix\_config/main.cf`) to verify or alter the subnets allowed to relay through the Mail Gateway (`mynetworks`), and define the internal authoritative mail routing:

```
—— Domains ———
# MGW does NOT accept final delivery – it only relays
mydestination      = localhost
mynetworks         = 127.0.0.0/8, 192.168.200.0/24, 10.10.0.0/24
relay_domains      = company.com

# — Relay outbound mail to the internal mail server —
transport_maps     = hash:/etc/postfix/transport
relayhost          =
```

2.2 Static Mail Routing Definition (`transport`)

If the downstream authoritative Mail Server utilizes an IP infrastructure or domain namespace variant from `10.10.0.20`, modify the `postfix_config/transport` map explicitly before building the image:

```
Route all company.com mail to the internal mail server
company.com      smtp:[10.10.0.20]:25
*                smtp:[10.10.0.20]:25
```

Note: Any runtime modifications to network boundaries or destination IPs require rebuilding the Docker image or executing `postmap /etc/postfix/transport` inside an active container session to rebuild the indexed routing databases.

3 Container Image Compilation and Dependency Resolution

To eliminate symlink fragmentation and localized virtual environment mismatches (`mail-env` and `venv`), the Dockerfile encapsulates a deterministic build process that instantiates clean python environments internally.

Compile the production-ready container image by executing:

Compile the container image while ignoring historical cache states

```
rama@mgw:~/Desktop/MGW-Docker$ sudo docker build -t shields-gate-mgw:v1 .
```

4. System Execution and Host-Network Integration

Because the Shields Gate system manages real-time MTAs, custom filter listeners (10025/10026), and deep sandbox API routing back to the core infrastructure, standard bridge network isolation can introduce detrimental routing abstractions.

To bridge the containerized processes seamlessly with the host's actual network interfaces, execute the container using the host network namespace:

Instantiate the containerized infrastructure with raw host-level network bindings

```
rama@mgw:~/Desktop/MGW-Docker$ sudo docker run -d \
> --name shields_gate_system \
> --network host \
> shields-gate-mgw:v1
[sudo] password for rama:
305749a92726834a284824a3ecf861d2178702f1a2c66a04b220db29e24f2cca
rama@mgw:~/Desktop/MGW-Docker$
```

5 Verification and Functional Integrity Mapping

To validate that the Mail Gateway, the NLP/ML processing server, and the CAPE Sandbox orchestrator have initialized cleanly without port collision or exit codes, execute the verification pipeline:

5.1. Verify container persistence and runtime status

```
rama@mgw: ~/Desktop/MGW-Docker$ sudo docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS          NAMES
4465675d2c8f   shields-gate-mgw:v1  "/entrypoint.sh"        9 seconds ago  Up 7 seconds  shields_gate_system
```

5.2. Inspect real-time process orchestration logs

```
rama@mgw: ~/Desktop/MGW-Docker$ sudo docker logs -f shields_gate_system
Cleaning up port 10025...
Starting Postfix...
...fail!
Starting CAPE Scheduler...
Starting CAPE Web API...
Starting MGW Mail Filter Server on port 10025...
/entrypoint.sh: line 15: /app/CAPEv2/venv/bin/python3: No such file or directory
/entrypoint.sh: line 19: /app/CAPEv2/venv/bin/python3: No such file or directory
2026-06-23 00:38:45,878 [INFO] MailFilter SMTP proxy listening on 0.0.0.0:10025
```

5.3. Confirm network socket binding across security-critical ports

```
rama@mgw: ~/Desktop/MGW-Docker$ sudo ss -tulpn | grep -E '25|587|10025|10026|8000'
[sudo] password for rama:
tcp        LISTEN     0          100        0.0.0.0:587      0.0.0.0:*    users:((("master",pid=1531,fd=109))
tcp        LISTEN     0          100        0.0.0.0:25       0.0.0.0:*    users:((("master",pid=1531,fd=106))
tcp        LISTEN     0          50         0.0.0.0:8000     0.0.0.0:*    users:((("python3",pid=1019,fd=12))
tcp        LISTEN     0          100        0.0.0.0:10025    0.0.0.0:*    users:((("python3",pid=23770,fd=8))
tcp        LISTEN     0          100        127.0.0.1:10026  0.0.0.0:*    users:((("master",pid=1531,fd=112))
```

Upon successful socket verification, the CAPE Sandbox management dashboard is exposed directly at <http://localhost:8000>, and the Mail Gateway is fully primed to intercept, analyze, and purge malicious email vectors.